

A separation logic for application placement

Stefano Forti

November 26, 2025

1 Model

Definition 1.1. A *placement state* is a pair $\sigma = \langle P, R \rangle$, where:

- $P : C \rightharpoonup N$ is a partial mapping of components $c \in C$ to nodes $n \in N$;
- $R : N \rightarrow \mathbb{N}$ gives, for each node n , the total capacity of n in terms of hardware units.

Definition 1.2. A placement state $\sigma = \langle P, R \rangle$ is *well-formed* iff:

$$\text{WF}(\langle P, R \rangle) \Leftrightarrow \forall c \in C. P(c) = m \in N \Rightarrow \delta(c, m) \wedge \forall n \in N. \sum_{c \in P^{-1}(n)} \rho(c) \leq R(n)$$

where

- $\delta(c, m)$ is a *deployment policy* that specifies the properties that node n must feature to suitably support the execution of component c , and
- $\rho(c)$ denotes the *hardware units* needed for the execution of component c

Definition 1.3. Let $\sigma = \langle P, R \rangle$ be a placement state. A *resource separation* of σ into two sub-states σ_1 and σ_2 , written

$$\sigma = \sigma_1 \oplus \sigma_2,$$

is defined as:

$$\sigma_1 = \langle P_1, R_1 \rangle, \quad \sigma_2 = \langle P_2, R_2 \rangle,$$

with

$$P = P_1 \dot{\cup} P_2 \quad \text{and} \quad \forall n \in N. R_1(n) + R_2(n) \leq R(n).$$

In words, each sub-state *owns* a disjoint portion of the available capacity of every node. Hence, node resources are locally partitioned between sub-placements.

This definition features compositionality by explicitly assigning each sub-state its own resource budget R_i . From a practical perspective, this corresponds to a *local* partitioning of node capacities. For instance, during continuous reasoning [1, 2, 3, 4, 5], the portion σ_{OK} keeps the resources already committed

to deployed components (its R_{OK}), while σ_{KO} operates over the overall infrastructure residual capacity $R_{KO} = R - R_{OK}$. As a consequence, reasoning about the validity of σ_{KO} becomes independent of σ_{OK} , thus enabling modular verification and efficient incremental recomputation. Particularly, as proved next, it is possible to check the well-formedness of each partitioned placement state independently from the other.

Lemma 1.1 (Compositional well-formedness). *If $\sigma = \sigma_1 \oplus \sigma_2$ according to the definition above, then*

$$\text{WF}(\sigma_1) \wedge \text{WF}(\sigma_2) \implies \text{WF}(\sigma_1 \oplus \sigma_2).$$

Proof. Assume $\text{WF}(\sigma_1)$ and $\text{WF}(\sigma_2)$. For any node $n \in N$,

$$\sum_{c \in P^{-1}(n)} \rho(c) = \sum_{c \in P_1^{-1}(n)} \rho(c) + \sum_{c \in P_2^{-1}(n)} \rho(c) \leq R_1(n) + R_2(n) \leq R(n).$$

Deployment policies δ trivially hold. Therefore $\text{WF}(\sigma)$ holds. $\square$

Definition 1.4. Given two assertions ϕ_1 and ϕ_2 holding over disjoint placement states σ_1 and σ_2 , their *separating conjunction*, written $\phi_1 \star \phi_2$, holds on the combined placement state $\sigma = \sigma_1 \oplus \sigma_2$ iff

$$\sigma_1 \models \phi_1 \quad \text{and} \quad \sigma_2 \models \phi_2.$$

2 Semantics

In classical separation logic [6], the *frame rule* enables local reasoning. It allows us to infer that an operation affects only the portion of the state it manipulates, while the rest of the state remains unchanged.

Formally, if an operation C transforms a placement state satisfying assertion ϕ into one satisfying ψ , then the same operation can be safely extended with any disjoint portion of the state χ that C does not alter (i.e. if C works on a separate portion of resources):

$$\frac{\{\phi\} C \{\psi\}}{\{\phi \star \chi\} C \{\psi \star \chi\}} \quad \text{if } \text{res}(C) \cap \text{res}(\chi) = \emptyset.$$

Under the compositional resource separation defined above, the $\star$ operator already guarantees that χ uses a distinct portion of the available resources. Hence, the rule ensures that any management operation – such as **deploy**, **undeploy**, or **migrate** (see next section) – only affects the subset of components and resources it operates on, while all other disjoint sub-states remain valid and unchanged.

The placement frame rule expresses the *locality of reasoning* in compositional placement semantics. Given a separation $\sigma = \sigma_1 \oplus \sigma_2$, an operation C acting only on σ_1 can be verified in isolation, and its correctness extends to the global state σ by framing out σ_2 .

In practice, this principle formalises the incremental behaviour of continuous reasoning. While σ_{KO} is updated through a sequence C^* of management operations (e.g., **deploy**, **undeploy** and **migrate**), the unaffected portion σ_{OK} is preserved and remains valid. In formulae, we can write the following:

$$\frac{\{ \neg \text{WF} \} C^* \{ \text{WF} \}}{\{ \neg \text{WF} \star \text{WF} \} C^* \{ \text{WF} \star \text{WF} \}} \quad \text{if } \text{res}(C^*) \cap \text{res}(\text{WF}) = \emptyset.$$

2.1 Management operations

We model deployment evolution as a state transition on placement states $\sigma = \langle P, R \rangle$ through three management operations: **deploy**, **undeploy** and **migrate**.

$$\frac{\text{DEPLOY} \quad c \notin \text{dom}(P) \quad \delta(c, n) \quad \sum_{x \in P^{-1}(n)} \rho(x) + \rho(c) \leq R(n)}{\langle P, R \rangle \xrightarrow{\text{deploy}(c, n)} \langle P[c \mapsto n], R \rangle}$$

If the component c is not yet deployed, the target node n satisfies the deployment policy $\delta(c, n)$, and n has at least $\rho(c)$ free capacity, then deploying c to n extends the placement P with $c \mapsto n$.

Lemma 2.1. *If $\text{WF}(\sigma)$ and $\sigma \xrightarrow{\text{deploy}(c, n)} \langle \sigma' \rangle$ by rule DEPLOY, then $\text{WF}(\sigma')$.*

$$\frac{\text{UNDEPLOY} \quad c \in \text{dom}(P) \quad P(c) = n}{\langle P, R \rangle \xrightarrow{\text{undeploy}(c)} \langle P \setminus \{c \mapsto n\}, R \rangle}$$

If component c is currently deployed on node n , the **undeploy** operation removes c from the placement P without altering the total node capacities R .

Lemma 2.2. *If $\text{WF}(\sigma)$ and $\sigma \xrightarrow{\text{undeploy}(c)} \langle \sigma' \rangle$ by rule UNDEPLOY, then $\text{WF}(\sigma')$.*

$$\frac{\text{MIGRATE} \quad c \in \text{dom}(P) \quad P(c) = s \quad \delta(c, d) \quad \sum_{x \in P^{-1}(d)} \rho(x) + \rho(c) \leq R(d)}{\langle P, R \rangle \xrightarrow{\text{migrate}(c, s, d)} \langle P[c \mapsto d], R \rangle}$$

If c is currently deployed to s , the node d satisfies the deployment policy $\delta(c, d)$, and d has enough free capacity w.r.t. the already placed components, then migrating c moves its placement from s to d .

Lemma 2.3. *If $\text{WF}(\sigma)$ and $\sigma \xrightarrow{\text{migrate}(c, n_s \mapsto n_d)} \langle \sigma' \rangle$ by rule MIGRATE, then $\text{WF}(\sigma')$.*

2.2 Infrastructure monitoring

Infrastructure partitions can evolve independently from explicit management operations, e.g. due to failures, node updates, hardware faults. We model such non-deterministic evolution through a MONITOR rule that can always trigger.

$$\frac{\text{MONITOR}}{\langle P, R \rangle \xrightarrow{\text{monitor}} \langle P, R' \rangle}$$

Note that monitor does not preserve well-formedness in general, since, for instance, capacity changes may invalidate previously admissible placements. However, it preserves resource separation, which is enough to retain the locality guarantees provided by the frame rule. Intuitively:

- **monitor** does not change P , as only management operations can directly act on application components, and
- it is always possible to determine a new resource separation R'_1 and R'_2 such that $R'_1 + R'_2 \leq R'$ (only existence is required by the definition).

References

- [1] FORTI, S., BISICCHIA, G., AND BROGI, A. Declarative continuous reasoning in the cloud-iot continuum. *Journal of Logic and Computation* 32, 2 (2022), 206–242.
- [2] FORTI, S., AND BROGI, A. Continuous reasoning for managing next-gen distributed applications. In *Proc. 36th International Conference on Logic Programming (Technical Communications)* (2020), vol. 325 of *EPTCS*, pp. 164–177.
- [3] HERRERA, J. L., BERROCAL, J., FORTI, S., BROGI, A., AND MURILLO, J. M. Continuous qos-aware adaptation of cloud-iot application placements. *Computing* 105, 8 (2023), 2037–2059.
- [4] HERRERA, J. L., BERROCAL, J., FORTI, S., BROGI, A., AND MURILLO, J. M. Multi-layered continuous reasoning for cloud-iot application management. *IEEE Transactions on Services Computing* 17, 6 (2024), 3985–4001.
- [5] MASSA, J., FORTI, S., DAZZI, P., AND BROGI, A. Combining declarative and linear programming for application management in the cloud-edge continuum. *Future Generation Computer Systems* 176 (2026), 108224.
- [6] O’HEARN, P. W. Separation logic. *Commun. ACM* 62, 2 (2019), 86–95.